

高精度加法

求 $a+b$ 之和， a, b 的位数小于等于1000位。

输入：

55555555555555555555
22222222222222222222

输出：

77777777777777777777

题目分析：以前我们学习的加法是直接输出 $a+b$ 就可以了，但是本题中 a, b 的位数都是小于等于1000位，而我们学习的最长的long long最大表示数也才16位左右，如果用来存储1000位的数肯定溢出了，所以我们应该换一种方法。

高精度加法

第一步：解决存储问题。

我们要计算出结果，肯定必须先把输入的两个数存储起来，单个变量肯定是没有办法存储的，那我们可以考虑用数组来存储，但是数字数组输入的时候只有分开输入才能表示单个数字，所以我们需要使用字符数组。

char s1[100]	‘1’	‘2’	‘3’	‘4’	‘5’	‘\0’
	s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]
char s2[100]	‘3’	‘6’	‘9’	‘\0’	‘\0’	‘\0’
	s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]

代码：scanf ("%s %s", s1, s2);

高精度加法

第二步：转换

我们存储问题解决后，还需要预处理一下，因为我们存储的时候用的是字符数组存储，里面的数字其实是数字字符，直接计算很不方便。

字符： '1' + '1' = 'b' (98对应的字符) ✗
数字： 1 + 1 = 2; ✓

char s1[100]

'1'	'2'	'3'	'4'	'5'	'\0'
s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]

int a[100]

1	2	3	4	5	0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]

高精度加法

char s2[100]

'3'	'6'	'9'	'\0'	'\0'	'\0'
s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]

int b[100]

3	6	9	0	0	0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]

代码:

```
for(int i=0;i<11;i++)  
a[i]=s1[i]-'0';  
for(int i=0;i<12;i++)  
b[i]=s2[i]-'0';
```

高精度加法

第三步：计算

当我们把字符数组转换成对应的数字数组后，我们就可以开始计算了，计算方法按照我们初学加法的时候用的小学加法来计算。个位加个位，十位加十位，逢十进一。

int a[100]

1	2	3	4	5	0
---	---	---	---	---	---

a[0] a[1] a[2] a[3] a[4] a[5]

int b[100]

3	6	9	0	0	0
---	---	---	---	---	---

a[0] a[1] a[2] a[3] a[4] a[5]

$$\begin{array}{r} 12345 \\ + \quad 369 \\ \hline 12714 \end{array}$$

$$\begin{array}{r} 12345 \\ + 369 \\ \hline 49245 \end{array}$$

for(int i=0;i<1;i++)
c[i]=a[i]+b[i];

高精度加法

第二步：转换

当我们把字符数组转换的时候，最好数组反向，这也符合我们的计算规则，先让最低位相加，再依次让高位相加。

char s1[100]	'1'	'2'	'3'	'4'	'5'	'\0'
	s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]
int a[100]	5	4	3	2	1	0
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
char s2[100]	'3'	'6'	'9'	'\0'	'\0'	'\0'
	s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]
int b[100]	9	6	3	0	0	0
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]

高精度加法

代码:

```
for(int i=0;i<11;i++)  
a[i]=s1[11-1-i]-'0'  
for(int i=0;i<12;i++)  
b[i]=s2[12-1-i]-'0';
```

int a[100]

5	4	3	2	1	0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]

int b[100]

9	6	3	0	0	0
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]

int c[100]

4	1	7	2	1	0
c[0]	c[1]	c[2]	c[3]	c[4]	c[5]

高精度加法

第三步：计算

规则：从最低位开始相加，逢十进一。

```
l=max(11,12);
for(int i=0;i<l;i++)
{
    c[i]=a[i]+b[i]+x;
    x=0; //进位
    if(c[i]>9)
    {
        c[i]-=10;
        x=1; //进一
    }
}
c[1]=x; //如果最高位继续进位
```

```
l=max(11,12);
for(int i=0;i<=1;i++)
{
    c[i]=a[i]+b[i]+x;
    x=0; //进位
    if(c[i]>9)
    {
        c[i]-=10;
        x=1; //进一
    }
}
```

注意x=0的1位置

两种写法有什么区别？

高精度加法

```
x=0;
for(int i=0;i<=1;i++)
{
    c[i]=a[i]+b[i]+x;
    x=c[i]/10; //大于9, x=1, 小于9, x=0
    c[i]=c[i]%10; //大于9, 减10, 小于9, 不变
    //不需要判断语句
}
```

注意：计算的时候是从低位往高位计算，并且，要注意n位数+m位数变成 $\max(n, m) + 1$ 位数这种情况。

高精度加法

第四步：去除多余前导零

int c[100]	4	1	7	2	1	0
	c[0]	c[1]	c[2]	c[3]	c[4]	c[5]

c[0]是最低位，c[5]是最高位，最高位上的数不能为0。
删除前导零的方法为，从最高位往最低位遍历，找到第一个不为0的数位置，并且记录位置。

```
for(int i=l-1;i>=0;i--)    for(int i=k;i>=0;i--)
{
    if(c[i]!=0)             printf("%d",c[i]);
    {
        k=i;
        break;
    }
}
```

那么问题来了！！！如果最终的结果就是0呢？结果会怎么样？

高精度加法

第一种:

```
k=0;
for(int i=l-1;i>=0;i--)
{
    if(c[i]!=0)
    {
        k=i;
        break;
    }
}
```

第二种:

```
for(int i=l-1;i>=0;i--)
{
    if(c[i]!=0||k==0)
    {
        k=i;
        break;
    }
}
```

```
for(int i=k;i>=0;i--)
    printf("%d",c[i]);
```

注意：这只是当数组的起点为下标0的时候的写法，如果数组的下标起点为1，则应该变为k=1；

高精度加法

```
int l1, l2, k, x=0;
scanf("%s\n%s", s1, s2);
l1=strlen(s1);
for(int i=0; i<l1; i++)
a[i]=s1[l1-i-1]-'0';
l2=strlen(s2);
for(int i=0; i<l2; i++)
b[i]=s2[l2-i-1]-'0';
for(int i=0; i<=max(l1, l2); i++)
{
    c[i]=a[i]+b[i]+x;
    x=c[i]/10;
    c[i]=c[i]%10;
}
```

```
for(int i=max(l1, l2); i>=0; i--)
{
    if(c[i]!=0 || i==0)
    {
        k=i;
        break;
    }
}
for(int i=k; i>=0; i--)
{
    printf("%d", c[i]);
}
return 0;
```

使用max函数的时候需要头文件
`#include<algorithm>`

高精度减法

$$\begin{array}{r} 14 \\ - 6 \\ \hline 8 \end{array} \quad \checkmark$$

$$\begin{array}{r} 6 \\ - 14 \\ \hline -92 \end{array} \quad \times$$

$$\begin{array}{r} 6 \\ - 14 \\ \hline -8 \end{array} \quad \checkmark$$

$$6 - 14 = - (14 - 6) = -8$$

第一步：用字符串存储（和加法一样）。

第二步：字符串转换成数字数组（和加法一样）。

高精度减法

第三步：比较大小，如果 $a < b$ 就交换 a, b 的位置，并且直接输出一个负号。

第一种：

直接通过比较字符串来比较大小(字符串未反向)。

```
x=strcmp(s1, s2);  
if(x<0)  
{  
    printf("-");  
    交换a, b两个字符串;  
}
```

反例：

111 23

```
if(11<12)  
{  
    printf("-");  
    交换a, b两个字符串;  
}
```

反例：

6 9

高精度减法

```
if(11<12)
{
    printf("-");
    交换a, b两个字符串;
}
else if(11==12&&strcmp(s1, s2)<0)
{
    printf("-");
    交换a, b两个字符串;
}
```

反例:

0000

23

这种写法在输入数据没有前导零的情况下是正确的，但是在输入数据有前导零的情况下是错误的，多余的前导零会导致某个数位数比实际情况多。

高精度减法

char s1[100]

'0'	'0'	'0'	'0'	'1'	'\0'
s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]

char s2[100]

'0'	'1'	'2'	'0'	'\0'	'\0'
s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]

char s1[100]

'1'	'0'	'0'	'0'	'0'	'\0'
s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]

char s2[100]

'0'	'2'	'1'	'0'	'\0'	'\0'
s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]



高精度减法

第二种：

通过比较数字数组来比较大小（数组已反向）。

char s1[100]	'0'	'0'	'0'	'0'	'1'	'\0'
	s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]
char s2[100]	'0'	'1'	'2'	'0'	'\0'	'\0'
	s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]
int a[100]	1	0	0	0	0	0
	s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]
int b[100]	0	2	1	0	0	0
	s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]

高精度减法

int a[100]

1	0	0	0	0	0
s1[0]	s1[1]	s1[2]	s1[3]	s1[4]	s1[5]

int b[100]

0	2	1	0	0	0
s2[0]	s2[1]	s2[2]	s2[3]	s2[4]	s2[5]

在数字数组中，从最高位开始比较，只要出现“>”“<”就结束比较。

为什么不可以直接把字符串反向然后进行比较：
因为在字符串中，空字符使用‘\0’（0）表示，而我们输入的时候，多余的前导零是用‘0’（48）表示，这两者是不相等的，所以位数少的仍然比较小，是错误的。

高精度减法

```
l = max(l1, l2);  
for(int i=l-1; i>=0; i--)  
//相当于给位数少的添前导零
```

```
{  
    if(a[i]>b[i])  
    {  
        flag=0; //第一个数  
        break;  
    }  
    if(a[i]<b[i])  
    {  
        flag=1; //第二个数
```

注意：交换a,b数组的时候，由于a,b均是数字数组，不能使用任何字符串函数！！！！

```
if(flag==1) //交换a, b数组  
{  
    printf("-");  
    for(int i=0; i<l; i++)  
        c[i]=a[i];  
    for(int i=0; i<l; i++)  
        a[i]=b[i];  
    for(int i=0; i<l; i++)  
        b[i]=c[i];  
}
```

相等的时候也是不需要交换的

高精度减法

第四步：计算

规则：从最低位开始相减，不够向前借一位。由于a肯定是大于等于b的，所以不需要担心最高位不够借。

int a[100]	5	4	3	2	1	0
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
int b[100]	9	6	3	0	0	0
	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]
int c[100]	6	7	9	1	1	0
	c[0]	c[1]	c[2]	c[3]	c[4]	c[5]

高精度减法

```
for(int i=0;i<l;i++)  
{  
    c[i]=a[i]-b[i];  
    if(c[i]<0)  
    {  
        c[i]+=10;  
        a[i+1]--;  
    }  
}
```

当该位上的数相减的时候，如果不够减，向前借一位，该位+10，前一位的被减数-1.

注意：因为两个数相减，位数只会越来越少，所以只需要减到最高位就可以了，不需要+1. 上面这种写法会破坏原来的a数组，如果后面还需要使用a数组则需要换一种写法。

高精度减法

第四步：去除多余前导零(和高精度加法一样)

```
for(int i=l-1;i>=0;i--)  
{  
    if(c[i]!=0||k==0)  
    {  
        k=i;  
        break;  
    }  
}
```

```
for(int i=k;i>=0;i--)  
printf("%d",c[i]);
```

高精度减法

```
int l1, l2, l, k, flag=0;
scanf("%s%s", s1, s2);
l1=strlen(s1); l2=strlen(s2);
for(int i=0; i<l1; i++)
    a[i]=s1[l1-i-1]-'0';
for(int i=0; i<l2; i++)
    b[i]=s2[l2-i-1]-'0';
l=max(l1, l2);
for(int i=l-1; i>=0; i--)
{
    if(a[i]>b[i])
    {
        flag=0; break;
    }
    if(a[i]<b[i])
    {
        flag=1; break;
    }
}
```

if(flag==1) // 交换a, b数组

```
{
    printf("-");
    for(int i=0; i<l; i++)
        c[i]=a[i];
    for(int i=0; i<l; i++)
        a[i]=b[i];
    for(int i=0; i<l; i++)
        b[i]=c[i];
}
```

从高位到低位判断数组的大小，即数的大小

高精度减法

```
for(int i=0;i<1;i++)
{
    c[i]=a[i]-b[i];
    if(c[i]<0)
    {
        c[i]+=10;
        a[i+1]--;
    }
}
```

```
for(int i=1-1;i>=0;i--)
{
    if(c[i]!=0||i==0)
    {
        k=i;
        break;
    }
}
for(int i=k;i>=0;i--)
{
    printf("%d",c[i]);
}
```


高精度乘法

第一步：用字符串存储

第二步：反向转换成数字数组

第三步：计算

```
c[i]=a[i]*b[i]+x;
```

```
x=c[i]/10;
```

```
c[i]=c[i]%10;
```



$$\begin{array}{r} 39 \\ \times 45 \\ \hline 195 \\ 156 \\ \hline 1755 \end{array}$$

$$\begin{array}{r} 39 \\ \times 45 \\ \hline 165 \end{array}$$



我们在使用乘法的时候，是让第一个乘数的**每一位**乘以**第二个乘数**的每一位。

所以我们应该使用**双重循环**结构分别遍历两个乘数的每一位。

高精度乘法

```
for(int i=0;i<11;i++)
{
    x=0;
    for(int j=0;j<12;j++)
    {
        c[?]=a[i]*b[j]+x;
        x=c[?]/10;
        c[?]=c[?]%10;
    }
}
c[i+j]=a[i]*b[j]+x;
x=c[i+j]/10;
c[i+j]=c[i+j]%10;
```

为什么不是 $i+j-1$?

$$\begin{array}{r} 39 \\ \times 45 \\ \hline 195 \\ 156 \\ \hline 1755 \end{array}$$

第一位	×	第一位	→	第一位
第二位	×	第一位	→	第二位
第一位	×	第二位	→	第二位
第二位	×	第二位	→	第三位
第 i 位	×	第 j 位	→	第 ? 位

最终结果第 k 位的值是所有 $i+j=k$ 的情况的值之和

高精度乘法

```
for(int i=0;i<11;i++)  
{  
    x=0; //进位  
    for(int j=0;j<12;j++)  
    {  
        c[i+j]=a[i]*b[j]+x+c[i+j];  
        x=c[i+j]/10;  
        c[i+j]=c[i+j]%10;  
    }  
    c[i+12]=x; //最高位进位  
}
```

当用第 i 个数乘完第二个数所有的数的时候，进位变为0.

最终结果第 k 位上的数，等于所有 $i+j=k$ 的情况的值之和

如果第一位从1开始的，该怎么写？

如果有当前计算的最高位向前进一位，应该把进位放在 $i+12$ 上面

高精度乘法

第四步：去除前导零

如果a数组有x位数，b数组有y位数，那么c数组最小开多大？

$$x + y$$

所以：c数组的大小应该开成a，b数组长度之和。

```
l=11+12;
for(int i=l;i>=0;i--)
{
    if(c[i]!=0||i==0)
    {
        k=i;
        break;
    }
}
for(int i=k;i>=0;i--)
printf("%d",c[i]);
```

高精度除法

输入两个整数a, b, 求a/b的结果, a为高精度大整数, b为低精度

第一步：存储

高精度数用字符串存储，低精度数用单个数字存储

第二步：字符串变数字数组

直接把字符串变成数字数组，注意除的时候是从最高位开始除，所以不需要反向存储

$$\begin{array}{r} 0205 \\ 6 \overline{) 1234} \\ \underline{12} \\ 03 \\ \underline{00} \\ 34 \\ \underline{30} \\ 4 \end{array}$$

高精度除法

第三步：计算

用被除数的每一位分别除以除数，得到的商用一个数组来存储，余数 $\times 10$ 和被除数的下一位相加，继续相除。

```
for(int i=0;i<l;i++)  
{  
    c[i]=a[i]/b; //商  
    t=a[i]%b; //余数  
    a[i+1]=t*10+a[i+1];  
}
```

大整数我们是按照输入的顺序存储的，所以第一位是最高位，最后一位是最低位。

第四步：去除多余前导零并输出存储商的数组

高精度除法

输入两个整数 a, b ，求 a/b 的结果， a 为高精度大整数， b 也为高精度大整数

当除数也是大整数的时候，即高精度除以高精度的时候，不可以直接用前面的办法直接相除了，因为除数也要用字符串数组来存储，所以要模拟大整数减法来计算。边比较边 $\times 10$ 相减。具体写法自己看书或者网上看博客。



高精度作业

洛谷，P1601, A+B

洛谷，P2142 高精度减法

